

计算理论导引笔记

2025

by kiwiizzz&xhkzdepartedream

Contents

1. 时间复杂性	2
1.1. 引入	2
1.1.1. 图灵机	2
1.1.2. 格局	2
1.1.3. 问题与语言	3
1.2. 时间可构造性	4
1.3. 通用图灵机	4
1.4. 对角线方法	6
1.5. 加速定理	7
1.6. 时间复杂性类	8
1.7. 非确定图灵机	8
1.7.1. 非确定图灵机的定义及其时间定义	8
1.7.2. P, NP, EXP, NEXP 问题	9
1.7.3. 快照	9
1.7.4. 通用非确定图灵机	9
1.8. 命题 & 谓词逻辑	9
1.9. 很多定理	10
2. Space Complexity	14
2.1. 格局图	14
2.2. 空间复杂性类	15
2.3. 空间谱系定理	16
2.4. Logspace Reduction	16
2.5. Space Completeness	17
2.6. NL 完全性	19
2.7. Immerman-Szelepcsenyi Theorem	19
3. NP Completeness	22
3.1. NP-难	22
3.2. Cook-Levin 定理	23
3.3. Berman-Hartmanis 猜测	25
3.4. Ladner 定理	26
3.5. 神谕图灵机,以及更多	27
3.5.1. 神谕图灵机的定义、Lowness	27
3.5.2. Cook Reduction	28

§1. 时间复杂性

§1.1. 引入

§1.1.1. 图灵机

Definition 1.1.1.1 (图灵机).

一台 k -带图灵机 M 是一个三元组 (Γ, Q, δ) , 其中:

1. 有限符号集 Γ , s.t. $\Gamma \supseteq \{0, 1, \square, \triangleright\}$;
2. 有限状态集 Q , s.t. $Q \supseteq \{q_{start}, q_{halt}\}$;
3. 迁移函数 $\delta: Q \times \Gamma^k \rightarrow Q \times \Gamma^{k-1} \times \{L, S, R\}^k$.

M 的第一条带子是只读的输入带, 剩下的都是工作带; 最后一条带子是输出带.

This course is about *classifying and comparing* problems by the amount of resource necessary to solve them.

— Turing, A. M.

Remark.

1. 有限状态集 Q 是机器所有可能“心理状态”的集合, 比如“正在读输入”“正在计算”“准备输出”等.
 - 必须包含: q_{start} 开始状态, 机器一启动就处于这个状态; q_{halt} 停机状态, 一旦进入这个状态, 机器就停止运行, 不再执行任何操作.
 - 你可以在 Γ, Q 中自己定义别的奇妙的符号, 但必须包含以上特别指出的符号.
2. 迁移函数输入: 当前状态 + k 个读写头各自看到的符号 (每条带子一个符号, 共 k 个) 输出:
 - 下一个状态 (Q 中的一个);
 - 要写到每条带子当前格子的新符号 (k 个符号 $\rightarrow \Gamma_k$);
 - 每个读写头下一步怎么移动: L, S, R 分别表示读写头向左、右或者保持不动.

□

Example.

图灵机的初始化:

1. $\triangleright$ 在每条带子的最左端;
2. 所有工作带的格子里都放空符号 $\square$;
3. 输入带除了输入之外的地方也是空符号.

我们自然的会把 $(q, a_1, \dots, a_k) \xrightarrow{\delta} (q', a'_2, \dots, a'_k, A_1, \dots, A_k)$ 称为一条指令. 其中 $A_i \in \{L, S, R\}$ 即一条移动; 根据定义, a_1 不需要修改, 因为是只读的.

§1.1.2. 格局

下面来看格局.

Definition 1.1.2.1 (格局).

$M(x)$ 表示预制输入 x . 计算的任意时刻 t 的格局 $\sigma_t = (q, \kappa_2, \dots, \kappa_k, h_1, \dots, h_k)$ 是一个 $2k$ 元组, 其中 κ_i 是工作带内容, $h_i \in \mathbb{N}$ 是读写头位置.

初始格局: $(q_{start}, \varepsilon, \varepsilon, \dots, \varepsilon, 0, 0, \dots, 0)$

种植格局: $(q_{halt}, \kappa_2, \dots, \kappa_k, h_1, \dots, h_k)$, 只要求 q_{halt} .

解读:

1. t 时刻的格局能反映这一时刻的图灵机状态,包含当前状态 q , $2 \sim k$ 条纸带的状态 $\kappa_2 \sim \kappa_k$,每条纸带上读写头的位置 h_i .
2. 初始格局是唯一的. 如果达到了终止格局,格局之间的传递被称为**计算路径**,详见 P6.
3. 时间函数记作 $T(n)$, $n = |x|$. $T(n) \geq n$,这是因为我们假定图灵机必须先完整的读一遍输入.
4. 可以用“快照”来理解“格局”,但“快照”在后面的部分仍有定义,不要混淆.

§1.1.3. 问题与语言

Definition 1.1.3.1 (问题与语言).

1. 一个函数 $f: \{0, 1\}^* \rightarrow \{0, 1\}^*$ 被称为一个**问题**.
 - 如果对每个 $x \in \{0, 1\}^*$, 图灵机 M 满足 $M(x) = f(x)$, 则称 M **计算或求解** 函数 f .
 - “ $M(x) = y$ ”表示“图灵机 M 在输入带预装 x 的情况下停机,且输出带上写有 y ”.
2. 一个函数 $d: \{0, 1\}^* \rightarrow \{0, 1\}$ 被称为一个**判定问题**.
 - 如果图灵机 M 计算 d , 则称 M **判定** d .

一个集合 $L \subseteq \{0, 1\}^*$ 被称为一个**语言**. 其中 L^* 表示 L 中元素的有限串集合.

- 如果图灵机 M 判定如下特征函数:

$$L(x) = \begin{cases} 1, & \text{如果 } x \in L \\ 0, & \text{如果 } x \notin L \end{cases} \quad (1)$$

那么称 M 接受语言 L , 或者 L 是可判定的.

- 一个判定问题 $A \subseteq \{0, 1\}^*$ 的补问题 A^c 定义为 $\{0, 1\}^* \setminus A$.

解读: 问题就是函数,函数就是问题. 参考书例子: 回文串 (朴素, P8).

一个问题是**图灵机可解的** $\iff$ 有一台图灵机计算它.

Theorem 1.1.3.1 (思考题).

为以下函数设计图灵机:

1. $s(x) = x + 1$. 用进位处理实现计数器.
2. $u(x) = 1^x$. 附加 1^x 限制运行步数.
3. $e(x) = 2^x$. 输出 2^x .
4. $l(x) = \log x$. 若 $x = 2^k$, 输出 $|x| + 1$, 否则输出 $|x|$.

Proof.

1. $s(a) = +1$. 先找到串的末尾, 然后做如下操作:
 1. 如果当前位是 1, 将其翻转为 0, 然后向左移动一位;
 2. 如果当前位是 0, 将其翻转为 1, 然后停机;
 3. 如果是 $\triangleright$, 改写为 1, 然后停机.
2. $u(a) = 1^n = \underbrace{11 \dots 11}_{n \uparrow 1}$. 对输入端做减法, 然后减 1 个就输出一个 1.
3. $e(a) = 2^n$. 用第二条给出 1^n , 然后对这个一元串, 读的第一个写 1, 读的后续所有都写 0.

4. $l(a) = \log(a)$. 先判断 x 的二进制有几位, 有几位就输出几个 1, 成为串 L ; 然后判断 x 是不是 2 的幂次, 如果第一个是 1 就往右挪, 如果碰到了另一个 1 就是 N, 否则是 Y; 最后, 如果是 Y 就在 L 后补一个 1, 如果是 N 就不做, 最后, 把 L 变成二元串.

□

§1.2. 时间可构造性

Definition 1.2.1 (时间可构造性).

一个函数 $T: \mathbb{N} \rightarrow \mathbb{N}$ (且 $T(n) \geq n$) 被称为是 时间可构造的 (Time Constructible) .

- 如果存在一台图灵机 M , 它能在 $O(T(n))$ 时间内计算并输出 $T(n)$ 的二进制表示 (即计算函数 $1^n \mapsto [T(n)]$) .

一个函数 $T: \mathbb{N} \rightarrow \mathbb{N}$ (且 $T(n) \geq n$) 被称为是 完全时间可构造的 (Fully Time Constructible) .

- 如果存在一台图灵机 M , 它在接收输入 1^n 后, 会在恰好 $T(n)$ 步后停机.

解读: 时间可构造性是指, $T(n)$ 本身可以被一个图灵机在接收 1^n (表示 n 这个数) 的输入后, 在有限步 ($O(T(n))$) 内, 输出 n 的二进制表示.

若在图灵机里面加入一个不停向右移动一格、到达规定位置后停机的条带, 就得到了一个 时钟图灵机 T 作为计时器, 用以限制另一台图灵机 (M) 的运行时间, 强制它在指定步数内停止.

Definition 1.2.2 (硬连接).

我们可以构造一台新的图灵机 M' , 它 (利用时间函数是 $T(n)$ 的时钟图灵机 T) 通过以下方式为 M 的计算强制设定一个时间上限:

1. 在输入 x 上, M' 首先计算出步数上限 $k = T(|x|)$. (由于 T 是时间可构造的, 此步骤可在 $O(T(|x|))$ 时间内完成) .
2. M' 模拟 M 在输入 x 上的运行, 最多 k 步.
3. 如果 M 在 k 步内停机, M' 就停机并返回 M 的结果. 如果 M 在 k 步内没有停机, M' 会强制停机并进入拒绝状态.

具体的构造如下:

$$\begin{aligned} M &= (Q_0, \Gamma_0, \delta_0), T = (Q_1, \Gamma_1, \delta_1), M' = (Q, \Gamma, \delta) \\ Q &= Q_0 \times Q_1, \text{ assuming } (q, q_{\text{halt}}^1) = (q_{\text{halt}}^0, q) = q_{\text{halt}} \\ \Gamma &= \Gamma_0 \times \Gamma_1 \\ \delta &= \delta_0 \times \delta_1 \end{aligned} \quad (2)$$

解读: 构造一台集成的图灵机 M' , 前 k_0 条给 M 用, 后 k_1 条给 T 用, 用笛卡尔积来定义 M' 的状态. 新机器的停机条件为: M 和 T 其中有任何一台停机.

一个图灵机被称为 神游的, 如果读写头的逻辑是确定的.

§1.3. 通用图灵机

Definition 1.3.1 (通用图灵机).

1. 一个转移规则 $(p, a, b, c) \rightarrow (q, d, e, R, S, L)$ 可以被编码为, 比如说:

$$001 \uparrow 1010 \uparrow 1100 \uparrow 0000 \uparrow\uparrow 011 \uparrow 1111 \uparrow 0000 \uparrow 01 \uparrow 00 \uparrow 10 \quad (3)$$

, $\uparrow$ 是不同字符之间的分隔符, $\uparrow\uparrow$ 表示状态编码和转移方向之间的间隔.

2. 一个转移表可以被编码为一个具有如下形式的字符串:

$$\ddagger \langle \text{rule}_1 \rangle \ddagger \langle \text{rule}_2 \rangle \cdots \ddagger \langle \text{rule}_m \rangle \ddagger \quad (4)$$

其中,二进制表示可以通过以下映射获得:

$$\begin{aligned} 0 &\rightarrow 01, \\ 1 &\rightarrow 10, \\ \uparrow &\rightarrow 00, \\ \ddagger &\rightarrow 11. \end{aligned} \quad (5)$$

解读:

就像程序可以作为数据被另一个程序读取和执行,这一部分的目标是:把一台图灵机的所有信息(状态、符号、转移规则)压缩成一个二进制字符串,这样它就可以被其他图灵机读取和模拟.

编码的一些性质:

- 每一台图灵机都可以表示为 01 编码的字符串,且编码是无限的,因为 $0^i \sigma 0^j \iff \sigma$;
- $\forall \Sigma \in \{0, 1\}^*$, 它都代表着一个图灵机.(即使有一些无意义的串,也将其强行联系某个垃圾图灵机.比如 0^n 根本编码不出 $\ddagger$, 所以他是非法的;基于此,我们把所有的非法串映射到一个特定的 M_0)

我们可以接着考察图灵机的枚举.根据上面的假定,我们可以建立一个双射 $f: \{0, 1\}^* \rightarrow \mathcal{TM}$, $\mathcal{TM}$ 是所有图灵机构成的集合.我们把它称作对图灵机的枚举.

用

$$\varphi(x) \triangleq \begin{cases} y, & M_{i(x)} = y \\ \uparrow, & M_{i(x)} \uparrow \end{cases} \quad (6)$$

来编码可计算的图灵机.

Theorem 1.3.1 (枚举定理,以及更多).

存在一个通用图灵机 U , 它使以下陈述为真:

1. 对于所有的 $x, \alpha \in \{0, 1\}^*$, 有 $U(x, \alpha) \simeq M_\alpha(x)$. 等价关系的定义是, 要么他们都停机且输出相同, 要么他们都在运行.
2. 如果 $M_\alpha(x)$ 在 $T(|x|)$ 步内停机, 那么 $U(x, \alpha)$ 在 $cT(|x|) \log T(|x|)$ 步内停机, 其中 c 是一个关于 α 的多项式.
 - 具有 $\mathcal{O}(T(n))$ 时间放大的版本出现于 1965 年.
 - 当前版本发表于 1966 年.

解读: 存在一台通用图灵机, 它可以模拟任何一台图灵机. 这是说: 对于任意的输入, 上述两台图灵机的行为完全等价(产生的输出完全相同或者同时永不停机); 代价是一点额外的时间.

Proof. 我们来构造一个满足的图灵机. 完整的证明在 P13-15, 给出思路如下:

1. 我们先把 U 的主工作带分成 k 层, 用一条工作带虚拟的模拟 k 条带(用存 k 元组的方法来解决); 我们把 0 处的位置记作读写头读取的东西, 然后我们把读写头的移动模拟为带子的反向移动.
2. 存放冗余信息(用 $\times$ 表示), 把主工作带划分为以 2^i 为长度的多个区间, 如图所示;

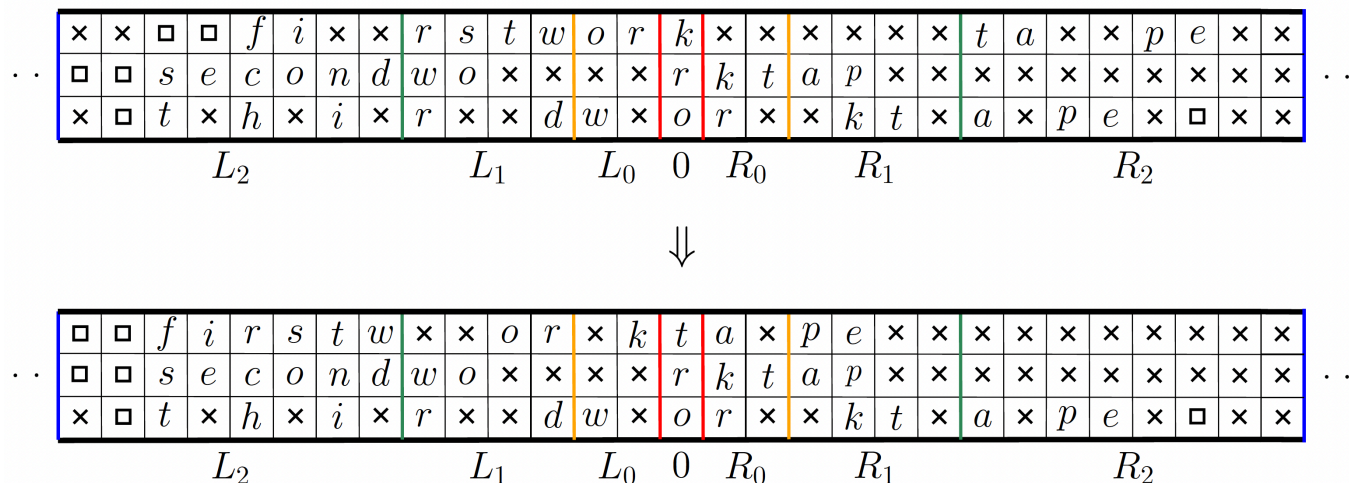


Figure 1: 证明图

3. 填充的规则如下: L_i, R_i 上填充的有效信息加起来是 2^{i+1} , 且单个是只有满、半满、空 3 种状态. 填充这个需要时间复杂度 $O(T \log T)$.
4. 来考察移动. 只考虑左移. 找到 i , s.t. $R_0, \dots, R_{i-1}$ 全空, 然后把 R_i 中的 2_i 个元素把前面 $i-1$ 个全部填充成半满, 同时替换 0 层; 然后, 原有的 L_{i-1} 中的有效信息填入 L_i 中, 以此类推. 图见上. 我们可以通过 U 的第二条工作带来复制碰到的第一个非冗余信息, 然后在区间大小的线性时间内完成复制工作. 这里的时间复杂度是

$$\sum_{i=0}^{\log T(n)} \left(2^i \cdot \frac{T(|x|)}{2^i} \right) = O(T(n)) \cdot \log(T(n)). \quad (7)$$

只需证明 $k \leq c(x)$. 由于 $k \leq |x|$, 这是显然的. □

解读:

- 我们还是要用 01 串来编码符号集合.
- In other words, once a shift with index i is performed, the next $2^i - 1$ shifts of that parallel tape only involves indexes less than i . yyl 补全这些
- 这句话说明了, 调整一次后, 调整的内容里面垃圾是足够多的, 我们的下面的操作不需要太复杂, 因为有足够多的垃圾等我们搬空. 这种设计方式大大减少了复杂性.

我们称一个图灵机是**健忘的**, 如果在计算的时刻我们不考察输入了什么, 而是考察输入的长度和操作了多少步.

修改 U 的定义: 预先将所有 L_i, R_i 区间半满, 然后引入计数器模拟步数. 如果第 i 位从 0 变成 1, 扫描 $0-i$ 的区间, 把 $L_j, R_j, j \in \{0, 1, 2, \dots, i-1\}$ 置为半满. 这样, 他就是健忘的, 那我们有如下推论:

Proposition 1.3.1 (健忘图灵机).

一些推论 thm: consequences 设 L 可在 $T(n)$ 时间内被图灵机判定, 那么, $\exists C T(n) \log T(n)$ 时间内可判定 L 的健忘图灵机, 其中 C 是常数.

设 $f \in \text{TIME}(T(n))$, 有双读写带的图灵机在 $O(T(n) \log T(n))$ 时间内计算 f , 有单读写带的图灵机在 $O(T(n)^2)$ 时间内计算 f .

§1.4. 对角线方法

不存在一段程序(图灵机), 使得其能够计算所有函数. 可以使用对角线法以及通用图灵机构造出这样的反例. 我们的目标是找到一个无法被图灵机计算的函数.

Problem 1.4.1 (decidable).

UC 问题:

$$UC(\alpha) = \begin{cases} 0, & \text{if } M_\alpha(\alpha) = 1 \\ 1, & \text{otherwise} \end{cases} \quad (8)$$

停机问题:

$$Halt(\alpha, x) = \begin{cases} 1, & \text{if } M_\alpha(x) \downarrow \\ 0, & \text{otherwise} \end{cases} \quad (9)$$

它们都不能被判定.

Proof.

1. UC 需要循环嵌套. 假定 UC 可以判定 UC , 那么,

$$UC(\ulcorner UC \urcorner) = 0 \iff UC(\ulcorner UC \urcorner) = 1 \quad (10)$$

(直接代入 UC 的定义就可以得到这里的等价关系, 从而导出了矛盾). (张小皓提出了这个反例在互换 UC 里 0 和 1 就失效了, 但事实上两者都不可判定.)

2. 利用反证, 假设存在这样的 M_H , 那么我们给出可以判断 UC 命题的图灵机的构造:

$$M_H(\alpha) = \begin{cases} 0, & M_H(\alpha, \alpha) = 1 \wedge M_\alpha(\alpha) = 1 \\ 1, & \text{otherwise} \end{cases} \quad (11)$$

这便给出了一个矛盾. □

解读: 1. 否定性命题的一般证明方法就是利用反证法, 其中对角线方法给了很好的例子.

§1.5. 加速定理

加速定理的动机在于思考: 对于一个问题, 有没有最好的算法解决它? 我们现在知道, 结论否定 (Blum's Speedup Theorem). 为了证明他, 我们不证明他. 转而看向线性加速定理.

Theorem 1.5.1 (Linear Speedup(线性加速定理)).

设图灵机 M 在 $T(n)$ 步内判定 L . 对任意 $\varepsilon > 0$, 有 在图灵机 M', M' 能在 $\varepsilon T(n) + n + 2$ 步内判定 L .

Proof. 我们来构造这个 M' .

- M' 的符号集 $\Gamma' \supseteq \Gamma^m$, 并且在 $n + 2$ 步内把原来的符号压缩 m 倍. 其中 2 指的是从 $\triangleright$, $\sqcup$ 浪费的两步.
- M' 的读写头回正, 需要 $\frac{n}{m}$ 步.
- M' 的读写头模拟 m 步操作, 至多需要 5 步.
- 则

$$T'(n) \leq n + 2 + \frac{n}{m} + \frac{5}{m}T(n) \leq n + 2 + \frac{6}{m}T(n) \quad (12)$$

- 我们取 $m = \frac{6}{\varepsilon}$ 即可. □

解读:

1. 你总能把原来的运行时间压缩到任意小的比例 (ε 很小), 只需要再加一点“启动开销” ($n + 2$). 核心思想是: 让新机器 M' 一次“看多格”, 而不是像老机器 M 那样一格一格慢慢挪.

2. M' 需要:

- 读取当前“超级格子”的内容 (包含 m 个原始字符) $\rightarrow 1$ 步
- 根据 M 的状态和当前字符, 查表决定下一步动作 $\rightarrow 1$ 步
- 修改“超级格子”中的某个字符 (如果需要) $\rightarrow 1$ 步
- 决定读写头往左/右移动:

- 如果向右移出当前“超级格子”,就要跳到下一个“超级格子”,并调整内部位置 $\rightarrow 1$ 步;如果向左移出当前“超级格子”,同理 $\rightarrow 1$ 步
- 更新状态 $\rightarrow 1$ 步

所以,最多需要 5 步 来完成一次“模拟 m 步中的一步”的操作.

当 $T(n) \gg n$ 我们有一个类似小 o 的思想:

Proposition 1.5.1.

设图灵机 M 在 $T(n) \gg n$ 步内判定 L . (或者我们记作, $T(n) = \omega(n)$) 对任意 $\varepsilon > 0$, 有 在图灵机 M' , M' 能在 $\varepsilon T(n)$ 步内判定 L . (这就是说, 线性项被忽略了.)

证明见作业§2.4.

§1.6. 时间复杂性类

Definition 1.6.1 (时间复杂性类).

A decision problem $L \subseteq \{0, 1\}^*$ is in $\text{TIME}(T(n))$, if there exists a TM that accepts L and runs in time $cT(n)$ for some $c > 0$.

解读:

$\text{TIME}(T(n))$ 是一个集合: 所有能在 $T(n)$ 时间内被图灵机解决的问题的集合.

基于线性加速定理, 我们不能定义问题的复杂性类, 只能定义解决问题的时间的复杂性类.

我们先提出了 P . 一个复杂性类是一个模型无关的问题类. 我们把所有具有多项式时间算法的问题认为是同一类, 即令

$$P = \bigcup_{c \geq 1} \text{TIME}(n^c) \quad (13)$$

以及

$$\text{EXP} = \bigcup_{c \geq 1} \text{TIME}(2^{n^c}) \quad (14)$$

然后我们提出了一个“明显的定理”(证明他!).

Theorem 1.6.1 (时间复杂性类).

我们称 $\text{co}T = \{\bar{A} \mid A \in T\}$, T 是一个时间复杂性类. $\text{co}T \subseteq T \Leftrightarrow \text{co}T \supseteq T \Leftrightarrow \text{co}T = T$

Proof. 有人托梦给我证明了.——YYL

□

§1.7. 非确定图灵机

§1.7.1. 非确定图灵机的定义及其时间定义

非确定图灵机的引入是为了解决验证问题(Validation Problem). 它不是物理上可实现的, 但是他很好玩.

What nondeterminism provides is the power of **guessing**.

非确定图灵机 N 指的是有两个迁移函数的图灵机, 它可以不确定地使用两者的策略之一. 换言之, 每一步可以有多个选择. 从初始状态出发, 其所有的可能路径会像二叉树不断向下延伸(类似《银河系漫游指南》的无尽概率跃迁方程), 每条路径可能终止, 也可能不停机. 非确定并不是随机.

Definition 1.7.1.1 (非确定图灵机的接受与拒绝,及其时间定义).

1. 设非确定图灵机 N , 它的每条计算路径(也就是一个 $\delta_i, i \in \{0, 1\}$ 的序列)试图构造一个存在性证明. 如果构造成功, 输入 1 并称这个终止格局叫接受格局; 类似的定义拒绝格局. 当输入 x 的时候, 若 N 有一条计算路径终止于接受格局, 称 N 接受 x , 并记作 $N(x) = 1$; 类似定义拒绝.
2. 对于每一个输入 x , 以及每一个“非确定性选择序列”, 机器 N 必须在 $T(|x|)$ 步内停机 (到达 q_h).

§1.7.2. P, NP, EXP, NEXP问题

类似 P , 我们有 $NP, NEXP$. 有一个明显的结论 (P27)

Theorem 1.7.2.1.

$$P \subseteq NP \subseteq EXP \subseteq NEXP. \quad (15)$$

Proof. 第一个和第三个包含关系很显然; 中间的包含关系是说, 非确定性图灵机 (NTM) 在多项式时间内能解决的问题, 可以用确定性图灵机在指数时间内模拟出来. 根据非确定性图灵机的定义, 这也是可以理解的. $\square$

§1.7.3. 快照

类似的考察通用性, 我们给出快照的定义:

Definition 1.7.3.1 (快照).

一个 $k+1$ 元组 $(q, a_1, \dots, a_k) \subseteq Q \times \Gamma \times \dots \times \Gamma$. 它包含了读写头指向的符号, 但是不包含读写头位置和剩余工作带的内容.

解读:

1. 基于一个给定的图灵机, 快照的大小是恒定的, 而不像格局的长度是 $O(T(|x|))$.

§1.7.4. 通用非确定图灵机

类似前文, 我们可以定义一个“通用”的非确定图灵机 V .

1. 它猜测 $N_\alpha(x)$ 的一个终止于接受格局的快照序列和一个同等长度的 0-1 串, 后者表示 N_α 在计算时做的非确定选择, 即 δ_0, δ_1 的一个. 猜测快照序列时, V 只考虑输入带上的符号, 忽略工作带上的符号; 工作带上的符号通过猜测得到.
2. V 需要多一条带来记录 N_α 的实际读写过程. 对每一条工作带, V 对之前猜测到的 01 串中的合法的快照带回去验证是否成功. 注意, 这个验证是逐带进行的.

解读:

1. V 的代价是它都不一定停机, 因为我们没办法确定快照序列的长度, 也不能确定它完全停机; 有一个办法是利用时钟图灵机 T 来强制让他停止.
2. 广告位招租
3. 验证的进行: 首先, N 也需要写入操作数这一个过程, 我们的快照也记录下了这个信息; 读写头的移动被 δ_i 确定了; 读的什么内容被快照决定了 (这就是确定了读/写了什么内容).

§1.8. 命题 & 谓词逻辑

关于命题和谓词逻辑的基本知识参考离散数学. 来看在图灵机下的新的定义.

Definition 1.8.1.

可满足性问题是所有可满足的合取范式的集合, 记作 SAT . k -合取范式中, 每个语句最多含有 k 个字; 这些合取范式的集合被记作 $kSAT$.

Proposition 1.8.1.**SAT $\in$ NP.****Proof.** 给定合取范式 $\phi(x_1, x_2, \dots, x_n)$, 一个多项式时间的非确定图灵机可以猜测一个指派, 然后验证之. $\square$

为了简化问题, 我们规定谓词逻辑的个体域是布尔域 $\{0, 1\}$, 并简写 $\exists x_1 \exists x_2 \dots \exists x_k$ 为 $\exists x_1 x_2 \dots x_k$ (当然, $\forall$ 也这么做).

Definition 1.8.2 (QBF).如下形式的前束范式被称为**量化布尔公式**:

$$Q_1 x^1 Q_2 x^2 \dots Q_k x^k \cdot \phi(x^1, \dots x^k) \quad (16)$$

其中, x^i 为一串变量, Q_i 为 $\forall, \exists, \forall, \dots$ 或者 $\exists, \forall, \exists, \dots$.

问题 QBF 是所有永真的量化布尔公式的集合. 当然也有 kQBF.

§1.9. 很多定理

我们来讨论对不同的函数 f, g , $\text{TIME}(f(n)) \subseteq \text{TIME}(g(n))$ 是严格的. 我们给出三个定理.

Theorem 1.9.1 (时间谱系定理).若函数 f, g 是时间可构造的, 而且 $f(n) \log f(n) = o(g(n))$, 则 $\text{TIME}(f(n)) \subsetneq \text{TIME}(g(n))$.**Proof.** (对角线方法) 我们来证明一个否定命题 $\text{TIME}(g(n)) \not\subseteq \text{TIME}(f(n))$.

构造反例 L : 输入 x , 在 $\mathbb{M}_x$ 计算 $g(|x|)$ 步; 如果完成模拟, 输出 $\mathbb{M}_x(x)$ 的反转, 否则输出 0. 由定义, $L \in \text{TIME}(g(n))$, 且 $\mathbb{D}$ 在时间 $g(n)$ 判定 L .

接下来证明不存在图灵机 $\mathbb{M}$, 能够在 $f(n)$ 时间内判定 L .假定 $L \in \text{TIME}(f(n))$, 且有 $M_z(z)$ 能判定.

我们取充分大的 z , 由 1.3.2, 我们知道 $g(|z|) \gg f(|z|) \log(|z|)$, $\mathbb{D}$ 可以模拟完运算. 因为两者都判定 L , $s\mathbb{D}(z) = \mathbb{M}_z(z)$; 同时, 按照输出结果我们有 $\mathbb{D}(z) = \overline{\mathbb{M}_z(z)}$, 这便是一个矛盾.

 $\square$

解读: 1. 由于 g 时间可构造, 所以 $\mathbb{D}$ 可以用一个 $\mathbb{T}_{g(n)}$ 的时钟图灵机掐表来实现. 也就是说,

$$\mathbb{D} \leftarrow \mathbb{T}_{g(n)} \wedge \lambda z. \mathbb{U}(z, z) \quad (17)$$

, $\wedge$ 表示掐表.

2. 证明过程中的反证法看起来不那么显然, 但是是基于一个直观的命题上的:

$$\text{TIME}(f(n)) \subset \text{TIME}(g(n)) \quad (18)$$

3. 我们为什么可以取这样的 $L \in \text{TIME}(f(n))$, $M_z(z)$ 判定之, 并且这里的 z 还能充分大? 其实, 我们是这么做的: 取这样的 z , s.t. $\mathbb{M}_z$ 判定 L 的时间函数 $T(n) \leq C f(n)$.

4. 对角线方法的图例:

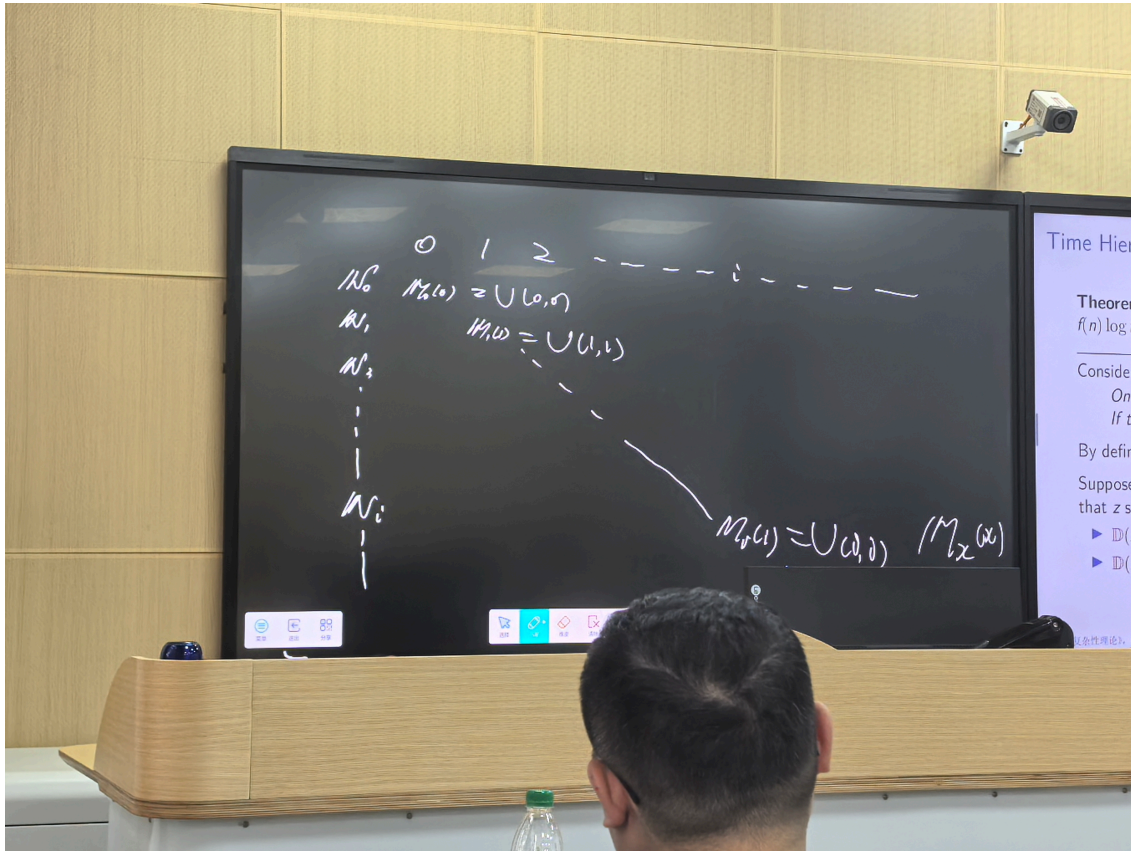


Figure 2: 对角线的来源是用通用图灵机模拟一个图灵机的时候,在对角线的时候取反.

初等函数类的定义是如下的:

$$\text{EXP} = \bigcup_{c>1} \text{TIME}(2^{n^c}) \quad (19)$$

$$2\text{EXP} = \bigcup_{c>1} \text{TIME}(2^{2^{n^c}}) \quad (20)$$

$$3\text{EXP} = \bigcup_{c>1} \text{TIME}(2^{2^{2^{n^c}}}) \quad (21)$$

$$\vdots \quad (22)$$

$$\text{ELEMENTARY} = \text{EXP} \cup 2\text{EXP} \cup 3\text{EXP} \cup \dots \quad (23)$$

非确定图灵机的谱系问题也有对应的定理和证明方法.这里的定理稍显奇怪(因为是 $f(n+1)$).

Theorem 1.9.2 (时间谱系定理).

若函数 f, g 是时间可构造的, 而且 $f(n+1) = o(g(n))$, 则 $\text{NTIME}(f(n)) \subsetneq \text{NTIME}(g(n))$.

Proof. 我们来设计一个精致的图灵机 Z 来证明这个命题.

- 首先, Z 的输入读写头和第一条工作带读写头往右扫描.
 - 若输入不形如 $1^n \wedge n > 1$, 直接停机并输出 0.
 - 第一条工作带上第一个位置写 1, 然后随机写 0 和 1. 我们记 $\{h_i\}$ 为工作带上写 1 的位置序列.
- 第二条工作带上, Z 直接模拟那些所有非确定图灵机和 $T_{2f(n)}$ 的硬连接. 不妨设这些图灵机为 $\{L_i\}$. Z 模拟完 $\{L_i\}$ 之后, 需要做如下操作来接着模拟 $\{L_{i+1}\}$:
 - 暂停第 1 步的操作;
 - 在第三条工作带上构造 $1^{h_{i-1}+1}$. 这是因为我们假定第三条工作带上保留着 $1^{h_{i-2}+1}$, 我们只需要再写 $h_{i-1} - h_{i-2}$ 个 1. 这里额外开销的时间是 $O(h_i)$ 是线性的;
 - 把 L_{i-1} 的编码复制到第三条工作带上. 这也是线性时间的;
 - 恢复第一步时候的读写头;

- 恢复输入带和第一条工作带读写头的同步全速向右扫描.与此同时, $\mathbb{Z}$ 用暴力法计算 $\mathbb{L}_{i-1}(1^{h_{i-1}+1})$.计算完后, $\mathbb{Z}$ 将结果写在第二条工作带上,与此同时,在第一条工作带上写 1,这个写了 1 的格子的地址就是 h_i .
3. 当输入带结束扫描之后,做如下操作:
- 若 $n = h_i$, $\mathbb{Z}$ 接受 $1^n \iff \mathbb{L}_{i-1}(1^{h_{i-1}+1}) = 0$;
 - 否则, $\mathbb{Z}$ 非确定的让 $\mathbb{V}(\mathbb{L}_{i-1}, 1^{n+1})$ 计算 $g(n)$ 步.

设 L 是 $\mathbb{Z}$ 接受的语言,来考察一下时间复杂度.关键性的是 $\mathbb{V}(\mathbb{L}_{i-1}, 1^{n+1})$ 的计算开销,不难看出其余步骤全是线性的;所以 $L \in \mathbf{TIME}(g(n))$.

另一方面,用反证法证明 $L \notin \mathbf{TIME}(f(n))$. 假设 $L \in \mathbf{NTIME}(f(n))$,且有 $\mathbb{N}_i$ 能判定.由线性加速定理,我们假定时间函数是 $2f(n)$.根据定义, $\mathbb{L}_i$ 接受 L ,那么当 i 充分大的时候,我们有

$$\begin{aligned}
 \mathbb{L}_i(1^{h_i+1}) &= \mathbb{Z}(1^{h_i+1}) \\
 &= \mathbb{L}_i(1^{h_i+2}) \\
 &= \mathbb{Z}(1^{h_i+2}) \\
 &= \dots \\
 &= \mathbb{L}_i(1^{h_{i+1}}) \\
 &= \mathbb{Z}(1^{h_{i+1}}) \\
 &\neq \mathbb{L}_i(1^{h_i+1})
 \end{aligned} \tag{24}$$

这一矛盾说明结论成立.

□

Remark.

- 为什么在 Step2 里,我们要用 $\mathbb{T}_{2f(n)}$ 来做计时工作?这是由于 Proposition 1.5.1 提出的线性加速定理的推论.
- 暴力法的意思就是枚举所有的计算路径
-

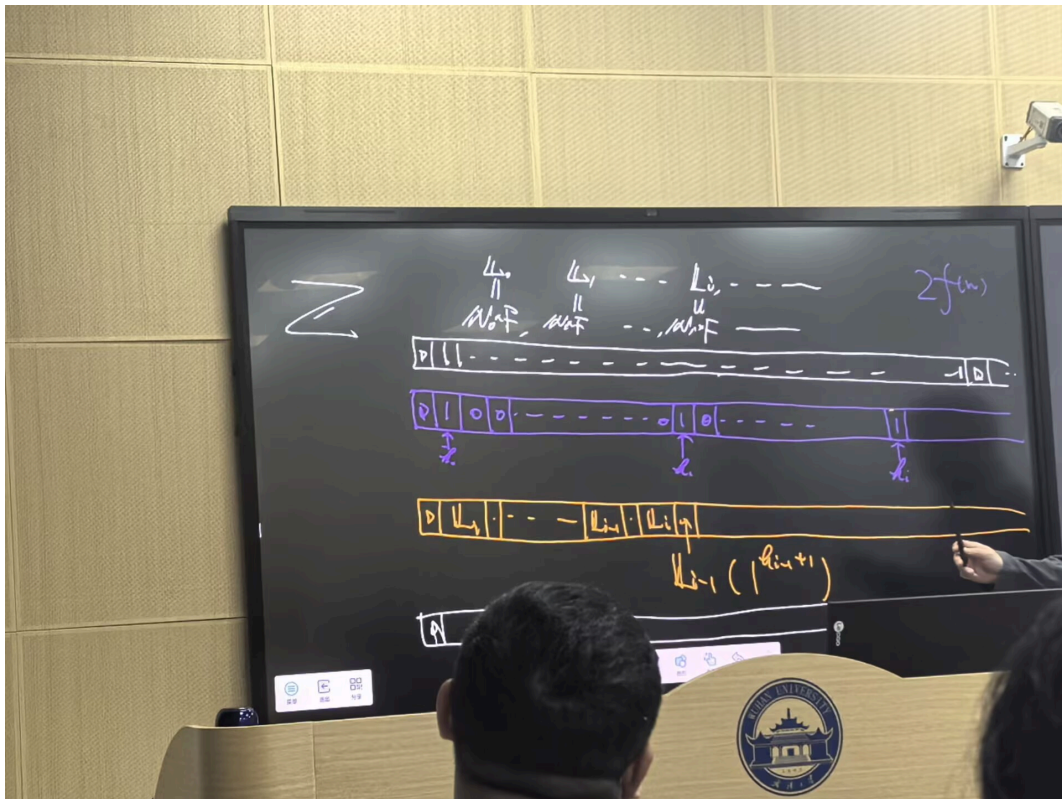


Figure 3: 这个图展示了每个带子上都写了些啥.从上到下分别是输入带,第一、第二条工作带,和草稿带.

- 既然 $\mathbb{L}_i$ 完全由递归呈现,那么 $\mathbb{L}_0$ 在哪里?注意,我们必须假定有一个成熟的算法来枚举 $\{\mathbb{L}_i\}$.并且我们不用关心时间复杂度.当 $\mathbb{Z}$ 的扫描输入这一步骤结束之后,枚举自然应该结束.

5.非确定图灵机里反转很麻烦,需要指数级的时间.所以这个证明绕来绕去的. □

作为本章的结束,来看一个定理.

Theorem 1.9.3 (间隙定理).

设 $r(x) \geq x$ 为可计算全函数 (不一定时间可构造的). 存在可计算全函数 $b(x)$, $\text{TIME}(b(x)) = \text{TIME}(r(b(x)))$.

Proof. 记 $k_0 < k_1 < \dots$, $k_0 = 0$, $k_{i+1} = r(k_i) + 1$. 并记 $P(i, k)$ 为“ $\forall j \leq i$, $|z| = i$, $\mathbb{M}_j(z)$ 要么 k 步内停机, 要么 $r(k)$ 步不停机”. 我们设 $n_i = \sum_{j=0}^i |\Gamma|^j$ 为图灵机 $\mathbb{M}_i$ 的符号串编码长度为 i 的总数, 讲这些符号串翻过去作为输入 (也就是对角线方法).

计算结果肯定不超过 n_i 个, 那么由抽屉原理, 存在 $j \leq n_i$, $P(i, k_j) = 1$. 定义 $b(i) = k_j$, 所以对所有 i , $P(i, b(i)) = 1$.

取 $L \in \text{TIME}(r(b(x)))$. 对足够大的 x , 总存在一个图灵机 $\mathbb{M}$, 它要在 $b(|x|)$ 步停机, 要么 $r(b(x))$ 步内不停机, 后者显然不符合, 这就说明了 $L \in \text{TIME}(b(x))$. □

Remark.

1. $b(x)$ 不是一个时间可构造函数, 因为间隙定理显然是与非确定时间谱系定理是有点矛盾的. □

§2. Space Complexity

可重用导致了空间复杂性的研究非常风格迥异.

Definition 2.1.

如果 $S: \mathbb{N} \rightarrow \mathbb{N}, L \subseteq \{0, 1\}^*$, 我们称 $L \in \mathbf{SPACE}(S(n))$, 如果存在一个 M 在 $cS(n)$ 个格子之内解决它.

与时间复杂性类似, 我们有空间复杂性的可构造性.

Definition 2.2.

如果 $S: \mathbb{N} \rightarrow \mathbb{N}, S(n) \geq \log(n)$, 我们称,

1. S 空间可构造 $\iff \exists M$, 它在 $O(S(n))$ 空间内把 1^n 转成 $\lfloor S(n) \rfloor$; 类似定义
2. S 完全空间可构造 $\iff$ 在上述过程中, M 刚好使用 $S(n)$ 空间.

对于非确定图灵机 N , 要求无论选择哪一条计算路径, 所用的工作带上格子数不超过 $O(S(n))$. 当输入

§2.1. 格局图

我们可以把图的可达路径问题和图灵机是否接受的问题挂钩. 格局图的用处立马可以被看到.

Definition 2.1.1.

1. 格局图 $G_{M,x}$ 的节点是 $M(x)$ 的格局, 有向边表示一步操作引起的合法的格局之间的转换.
2. 用 C_{start}, C_{end} 表示起始/终止格局, 我们就把 " M 是否接受 x " 等价成了 " $G_{M,x}$ 是否包含从 C_{start} 到 C_{end} 的路径".

格局图的一个点可以用长度为 $O(S(|x|))$ 的 01 串表示, 所以这个图顶多有 $2^{O(S(|x|))}$ 个顶点.

Remark. Configuration (格局) 的编码长度

一个格局 C 必须完整地编码图灵机的所有信息, 包括:

磁带内容: 共有 $S(|x|)$ 个磁带格子, 每个格子用固定的位数 (通常是 $O(1)$ 或 $O(\log|\Sigma|)$) 编码. 总长度为 $O(S(|x|))$.

读写头位置: 共有 $S(|x|)$ 个可能位置. 需要 $\log(S(|x|))$ 位来编码.

机器状态: 状态集合 Q 是有限的, 需要 $O(1)$ 或 $O(\log|Q|)$ 位来编码.

因此, 一个完整的格局 C 作为一个布尔变量串, 其总长度是由磁带内容长度主导的, 即:

$$\text{Length}(C) = O(S(|x|)) + O(\log S(|x|)) + O(1) = O(S(|x|)) \quad (25)$$

□

利用格局图我们可以解决如下问题:

Problem 2.1.1.

任意给你两张快照 C 和 C' , 我们如何快速验证是否能从 C 的状态, 根据规则, 只走一步就变成 C' 的状态?

这个问题的结果是惊讶的: 我们可以在 $O(S(n) \log(S(n)))$ 的时间内完成.

Proof. 设 $\kappa = \langle q, h, \tau_1, \dots, \tau_S \rangle$ 表示格局, q 是状态, h 是读写头位置, 后者全是工作带内容. 设状态 q 的编码长度为 Q : 假设有 Q 个状态, 我们就用 Q 个布尔变量 $u_1, u_2, \dots, u_Q$ 来表示. 设符号编码 x 长度为 e : $x_i = \{x_i^1, \dots, x_i^e, x_i^h\}$ (x_i^h 表示读写头是否指向 i) 定义一公式 $\varphi(u, x, v, y)$:

$$\varphi(u, x, v, y) = (x_i^h \wedge \phi(u, x_i, v, y_i, v, y_i, y_{i-1}^h, y_{i+1}^h)) \vee (\overline{x_{i-1}^h} \wedge \overline{x_i^h} \wedge \overline{x_{i+1}^h} \wedge (x_i^1 = y_i^1) \wedge \dots \wedge (x_i^e = y_i^e)) \quad (26)$$

其中, u 与 x, v 与 y 分别构成两个格局; ϕ 是一个根据迁移函数定义的合取范式.

定义

$$\varphi_{M,x} = \bigwedge_{i \in S(n)} \varphi_i \quad (27)$$

,这个公式的长度是 $O(S(n) \log(S(n)))$.

所以,我们可以对 $\varphi_{M,x}(C, C')$ 扫描一遍后就能求值,进而解决问题. $\square$

利用 bfs 的思想,我们可以得到这个定理的第三个包含.

Theorem 2.1.1.

$$\mathbf{TIME}(S(n)) \subseteq \mathbf{SPACE}(S(n)) \subseteq \mathbf{NSPACE}(S(n)) \subseteq \mathbf{TIME}(2^{O(S(n))}) \quad (28)$$

§2.2. 空间复杂性类

四个关键的定义.

$$\begin{aligned} \mathbf{L} &\stackrel{\text{def}}{=} \mathbf{SPACE}(\log(n)), \\ \mathbf{NL} &\stackrel{\text{def}}{=} \mathbf{NSPACE}(\log(n)), \\ \mathbf{PSPACE} &\stackrel{\text{def}}{=} \bigcup_{c>0} \mathbf{SPACE}(n^c), \\ \mathbf{NPSPACE} &\stackrel{\text{def}}{=} \bigcup_{c>0} \mathbf{NSPACE}(n^c). \end{aligned} \quad (29)$$

任何能在多项式时间内运行的(非确定性)计算,最多只能使用多项式空间.因为图灵机每一步最多使用一个新格子,所以运行 $T(n)$ 步最多使用 $T(n)$ 个格子.

我们知道 $\mathbf{NP} \subseteq \mathbf{PSPACE}$. 这是因为,空间可重用,而空间复杂性类里我们对时间毫无约束. 对非确定图灵机的不同的计算路径,我们可以先算一个 $(O(S(n)))$,再擦掉,最后再计算一次.

Remark. Games are Harder than Puzzles.(ppt)

Fuck 计算理论导引.(——YYL) $\square$

还有一个定理.

Theorem 2.2.1.

$$path \in \mathbf{NL} \quad (30)$$

PATH 问题: 给定一个有向图 $G = (V, E)$, 以及两个顶点 s (起点) 和 t (终点), 判断是否存在一条从 s 到 t 的有向路径.

Proof. 标准的确定性算法(如 BFS 或 DFS)需要线性空间($O(n)$)来记录已访问的节点,防止循环.但在非确定性模型中,我们可以“聪明地猜”一条路径,而不需要记录所有访问过的节点.

对任意一个点 $e \in G$, 设定一个非确定图灵机 N , 它猜测 e 的一个邻居(不妨为 f). 猜测成功后, N 把 e 改成 f , 继续猜测. 猜测 $|G|$ 次后, 如果不到达 t , 就拒绝. 一次操作自然是 $\log(|G|)$ 的, 工作带只需同时存储:

当前节点 v : $O(\log(n))$

计数器 $i : O(\log(n))$

总共: $O(\log(n)) + O(\log(n)) = O(\log(n))$ 空间. 那么, $\text{path} \in \mathbf{NL}$. □

§2.3. 空间谱系定理

nobody cares.

§2.4. Logspace Reduction

即使你不直接算出整个 $f(x)$, 只要你能“按需访问”它的每一位, 并且只用很少的空间 (对数空间), 那这个函数就算‘隐式对数空间可计算’.

Definition 2.4.1.

称 f 为隐式对数空间可计算的, 如果他满足:

1. $\exists c, \forall x, |f(x)| \leq c |x|^c$,
 2. $\{(x, i) \mid i \leq |f(x)|\} \in \mathbb{L}$
 3. $\{(x, i) \mid f(x)_i = 1\} \in \mathbb{L}$.
- (31)

Remark. 即使你不直接算出整个 $f(x)$, 只要你能“按需访问”它的每一位, 并且只用很少的空间 (对数空间), 那这个函数就算‘隐式对数空间可计算’.

1. 输出长度不能太长, 最多只能是输入长度的多项式倍数.
 2. 给定输入 x 和一个位置 i , 你能用对数空间判断: $f(x)$ 的长度是否至少有 i 位.
 3. 给定输入 x 和一个位置 i , 你能用对数空间判断: $f(x)$ 的 i 位是不是 1.
-

Definition 2.4.2.

可计算全函数 $f: \{0, 1\}^* \rightarrow \{0, 1\}^*$ 是问题 A 到问题 B 的 **m-归约**, 记为 $A \leq_m B$, 若满足: $\forall x \in \{0, 1\}^*, f(x) \in B \iff x \in A$.

我们称 B 是对数空间规约到 C 的, 当且仅当 B 到 C 的规约是隐式空间可计算的. 对数空间规约记作 $\leq_L$.

自然想讨论 $\leq_L$ 的结合律. 答案是肯定的.

Theorem 2.4.1.

$\leq_L$ 是结合的.

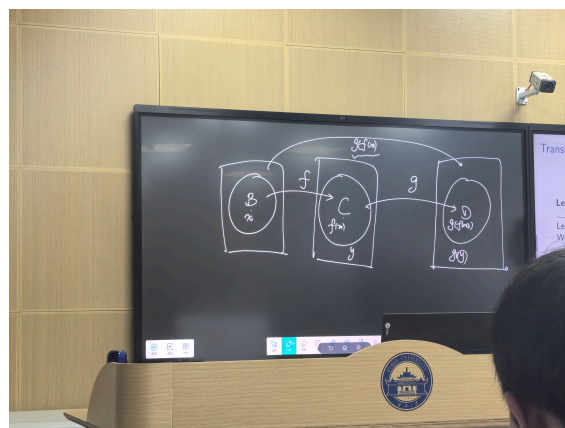


Figure 4: 派 YYL 来做这个漂亮的手写图

Proof. 我们维护一个对数长的计数器,来看 g 在处理 $f(x)$ 的哪一位.这是因为,图灵机在模拟 g 的时候,不需要记录 $f(x)$ 的每一位,而只关心读写头到了哪里.我们可以视作 $f(x)$ 被记录在了一个虚拟带中,靠这个计数器来模拟读写头的读写.

当 g 运行时,它的读写头会在虚拟带上移动.我们需要知道: g 的读写头当前在虚拟带的哪个位置? 这个位置可以用一个计数器来记录.因为 $f(x)$ 的长度是 $O(n^c)$ (多项式),所以计数器只需要 $O(\log n)$ 位.维护一个计数器 pos ,表示 g 的读写头当前在虚拟带 (即 $f(x)$) 的哪个位置.初始时, $pos = 0$.每次 g 需要读取虚拟带的 pos 位时,调用 f 的“按需访问”功能,计算 $f(x)_{pos}$.这可以在 $O(\log n)$ 空间内完成 (因为 f 是隐式对数空间可计算的).每次 g 移动读写头 (左移或右移),我们就更新 pos . $\square$

事实上,我们可以更直观的定义函数规约.以下的定义和 1.4.1 等价.

Definition 2.4.3.

称 $f: \{0,1\}^* \rightarrow \{0,1\}^*$ 是对数空间可计算的, iff, M_f 使用了工作带上的对数个格子,并且他的输出带是只写的: 也就是说,输出带的读写头每写一次都必须往右移一位.

Remark. tmd 为什么书上不证明这两个定义是等价的? $\square$

§2.5. Space Completeness

“...PSPACE-难的问题是对人类计算能力的极限挑战;我们已经无奈地接受了一个事实,就是人类是完全打不过多项式空间图灵机的.”

我们来考察一些问题的困难性.

Definition 2.5.1.

称 A 是 X -难的, iff, $\forall A \in X, A \leq_X B$. 如果 $A \in X$, 那么 A 是 X -完全的.

例如: A language L' is PSPACE-hard if $L \leq_{\log} L'$ for every $L \in \text{PSPACE}$.

If in addition $L' \in \text{PSPACE}$ then L' is PSPACE-complete.

PSPACE 中的每一个问题 L ,都可以用对数空间归约到 L' .也就是说, L' 至少和 PSPACE 中最难的问题一样难.但 L' 本身不一定在 PSPACE 中!

QBF 之前已经写过.我们来考察这一个不错的结果.

Theorem 2.5.1.

QBF 是 PSPACE-完全的. (Stockmeyer-Meyer 1974)

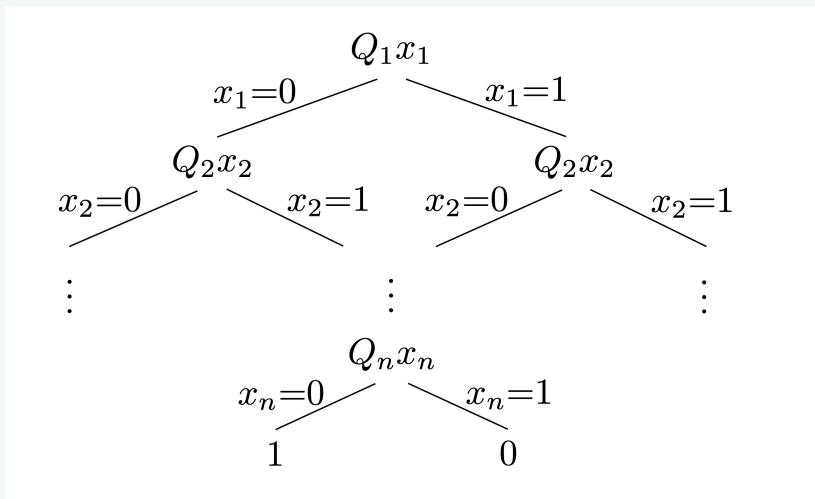
QBF: $Q_1x_1Q_2x_2\cdots Q_nx_n.\varphi(x_1,\dots,x_n)$, 其中 Q_i 是 $\forall$ 或者 $\exists$.

Proof. 先来看一个引理.

Lemma 2.5.1.

QBF 可在线性空间内判定.

Proof. 把 $\psi = Q_1x_1\cdots Q_nx_n.\varphi(x_1,\dots,x_n)$ 化成如下所示的二叉树:



然后,对其利用 dfs,得到一个指派,然后再线性空间里计算 ψ 的值. □

回到原定理.我们只需证 $\forall L \in \mathcal{L}, x \in \{0, 1\}^*, M$ 在 $S(|x|)$ 空间里判定 L .我们的想法是,把格局图中 $C_{\text{start}} \rightarrow C_{\text{accept}}$ 的路径用 QBF 表示.我们归纳的构造这样的 QBF.

我们令 $\psi_i(C, C')$ 判定是否有路径从 C 到 C' ,并且长度不超过 2^i .所以 $\psi_i(C, C')$ 的逻辑结构如下:

$$(\exists C \forall D^1 \forall D^2 ((D^1 = C \wedge D^2 = F) \vee (D^1 = C \wedge D^2 = C')) \Rightarrow \psi_{i-1}(D^1, D^2)) \quad (32)$$

上面一个公式的意思是: $\exists C''$: 先猜一个中间点 C'' .

$\forall D^1 \forall D^2$: 对于任意的一对节点 (D^1, D^2) ...

$(...) \Rightarrow \psi_i(D^1, D^2)$: 如果这对节点满足特定条件: 如果你考察的是 (起点, 中点) 这一对,或者 你考察的是 (中点, 终点) 这一对,那么它们之间必须连通 (调用 ψ_i).

再来估算 $|\psi_i|$.不难看出 $|\psi_i| = |\psi_{i-1}| + O(S(|x|))$.所以 $|\psi_x| = O(S(|x|)^2)$.所以我们定义了一个规约. □

Remark.

1. Why $|\psi_i| = |\psi_{i-1}| + O(S(|x|))$? 这是因为,的式子中,我们得利用蕴含恒等式去计算.蕴含的前面有四个等号,我们通过暴力枚举每个点来判断等号是否成立,这是 $O(S(|x|))$ 的.
2. 我们可以把一个 QBF 看成一个博弈游戏,A 先玩,B 后玩.那么 QBF 表示的意思自然就是 A 是否有获胜策略.上面这个式子就证明了,判断博弈游戏里一个人是否有获胜策略是 **PSPACE**-完全的.
3. 核心公式这一步不是把代码复制粘贴两遍 (导致代码膨胀),而是写了一个 for 循环或者定义了一个函数,然后传入不同的参数调用它.如果 D^1, D^2 不是 C, C', c 之一,上述式子一定真,因为前提是假的;当它们落进去了,就得递归的判断了.

□

因为格局图的大小是 $2^{S(|x|)}$,所以找到的中间格局会使得递归深度除以 2,也就是变成 $2^{S(|x|)-1}$,所以一共递归 $S(|x|)$ 次,所以复杂度是 $O((|x|)^2)$.

我们可以把定理的证明思路归纳成:利用 $\mathbb{N}$ 的猜测能力来解决 $\exists$,用计数器的枚举能力解决 $\forall$.下面这个定理给出了类似的思想.

Theorem 2.5.2 (Savitch Theorem).

如果 S 是空间可构造的,那么, $\mathbf{NSPACE}(S(n)) \subseteq \mathbf{SPACE}(S(n)^2)$.

Proof.

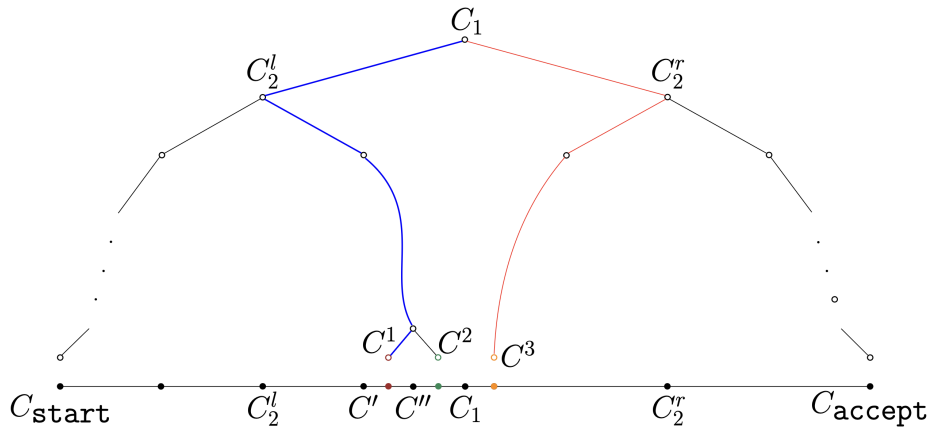


Figure 6: 证明的图

这张图直接展示了证明的思路:我们构造这样的格局二叉树,使得 $C_{\text{start}}, C_{\text{accept}}$ 在左下角和右下角.我们通过“先左儿子,再右儿子,再祖先”的顺序 dfs 整个二叉树,利用枚举二进制编码来判断是否存在右儿子.这个流程显然是确定图灵机可完成的.

那么,这里需要多少空间?二叉树的高度是 $O(\log(2^{S(n)})) = O(S(n))$ 的.一个格局的枚举是 $O(S(n))$ 的.由于最坏情况我们需要存 $C_{\text{start}} \rightarrow C_1$ 的所有路径,也就是说需要遍历所有层.所以,这个图所占的空间是 $O(S(n)^2)$.

□

这个定理的推论是重要的: $\text{PSPACE} = \text{NPSpace}$.左边包含右边是显然的.右边包含左边利用定义证明.注意到 $\text{NSpace}(p^n) \subseteq \text{Space}(p^{2n}), \forall p \in \text{poly}$.

§2.6. NL 完全性

Definition 2.6.1.

称 B 是 **NL-完全**的, iff, $\forall A \in \text{NL}, A \leq_L B$.

我们可以看到一个很惊讶的结果:如果 $A \in \text{NL}$,那么 A 是 **NL-完全**的. 我们通过证明 PATH 是 **NL-完全**的来证明这个结果.

Theorem 2.6.1.

PATH 是 **NL-完全**的.(Karp 1972)

Proof. 记 $L \in \text{NL}$.我们来证明 $L \leq_L \text{PATH}$.定义规约 $\gamma: x \mapsto \langle G_{N,x}, C_{\text{start}}, C_{\text{accept}} \rangle$.注意,我们可以用邻接矩阵储存 G , G 的每一个元素都可以在对数空间内计算得到.所以 γ 是对数规约.得证. □

§2.7. Immerman-Szelepcsenyi Theorem

在 Savitch Theorem 中,我们证明 $\text{CONSPACE} = \text{NPSpace}$.但是,coNL 和 NL 无法判断.

但是,Immerman 提出了,PATH 的补问题:

Theorem 2.7.1 (Immerman-Szelepcsenyi Theorem).

$\overline{\text{PATH}} \in \text{NL}$.

Proof. 用 bfs.记 s 为起点, t 为终点.我们引入集合 C_i, c_i, C_i 是所有能从 s 在 i 步内到达的点的集合, $c_i = |C_i|$.我们可以把一些(有限个) c_i 放到带子上,但是我们不能存哪怕任何一个 C_i . 具体的算法如下.

1. 记 $c_i = 0, i = 1, \dots, n$;
2. 计算 c_1 ;
3. 从 $c_i \rightarrow c_{i+1}$, 此时 c_i 的值是确凿无疑的, 计算过程如下:
 - 对每个不为 s 的节点 v , 若 $v \in C_i$ 或从某个 C_i 中的节点 u 到 v 有一条边, 则计数器 $c_{i+1} + 1$.

可是 C_i 无法被保存, 只能猜测:

- **猜测** C_i 这个点集, 猜测的结果被记为 $\hat{C}_i$, 并验证 $\hat{C}_i$ 是否等于 C_i :
 - 首先, $\hat{C}_i$ 之中的每个元素都和 s 的距离不超过 i , 我们用 PATH 算法验证.
 - 若 PATH 验证通过, 则还需要 $|\hat{C}_i| = c_i$. 这通过维护计数器可以做到.
 - 以上两步有任意一个条件未满足的图灵机 N 所在的时空分支停机, 而有一个时空中, 以上两步都通过, 则可以认为 $\hat{C}_i$ 是确凿无疑的了.
4. 最终图灵机已经正确地算出 c_{n-1} , 现在要判定 t 是否在可能的 C_{n-1} 里. 为此, 再猜测一次 C_{n-1} 即可.
- 非确定算法总会有一条计算路径将 C_i 正确猜出, 将 c_i 正确算出. □

Corollary 2.7.1.

$\text{coNL} = \text{NL}$.

Proof. PATH 是 **NL**-完全的, 所以 $\overline{\text{PATH}}$ 是 **coNL**-完全的. 又由 $\overline{\text{PATH}} \in \text{NL}$, 有 $\text{coNL} \subseteq \text{NL}$. 再由补问题的定义, 得证. □

Theorem 2.7.2.

2SAT 是 **NL** 完全的.

Proof.

1. 证明 $2\text{SAT} \in \text{NL}$ (相对简单).

假设我们有一个 2SAT 是 φ .

对 φ 的 k 个变量, 构造 $2k$ 个点 $x_1, \dots, x_k, \bar{x}_1, \dots, \bar{x}_k$; 构造边 $\bar{u} \rightarrow v, \bar{v} \rightarrow u \iff u \vee v \in \varphi$.

证明: φ 不可满足当且仅当 $v, \bar{v}$ 是通路.

必要性很显然, 充分性: 假定不存在 $v, \bar{v}$ 的通路, 我们对没有赋值的 $v, \forall w, s.t. v \rightarrow w$, 给 w 赋值为真, $\bar{w}$ 赋值为假. 这样归纳的我们找到了真值指派, 矛盾.

这就把 2SAT 问题归约到了图的不可达性问题 ($\overline{\text{PATH}}$), 而 ($\overline{\text{PATH}}$) 和 PATH 都是 **NL**-完全的;

所以 $2\text{SAT} \in \text{NL}$.

2. 证明 2SAT 是 **NL**-难的. 这个证明极其冗长(惩罚你去书上找推论 1.4)

Lemma 2.7.1.

有向无环图的可达性问题是 **NL**-完全的.

Proof. 如果我们把一个对数空间的非确定图灵机和一个多项式时间的时钟图灵机硬连接, 我们得到了一个每条计算路径都终止的图灵机. 它的格局图是有向无环图. 计算这个可达性问题自然就和计算图灵机的计算路径一致. 所以它是 **NL**-难的, 进而完全的. □

记 G 是一个有向无环图. 我们构造规约 $f: G \mapsto 2\text{SAT}$ 来说明命题. 对 $s, t \in V$,

$$\varphi_G \triangleq \{s \vee \bar{t} \vee \bigvee (\bar{x} \vee y) \mid \langle x, y \rangle \in E\} \quad (33)$$

. 那么 $\varphi_G \in 2\text{SAT}$, 并且容易验证这就是个规约, 命题得证 □

Theorem 2.7.3.

$$L \subseteq \text{NL} \subseteq \text{P} \subseteq \text{NP} \subseteq \text{PSPACE} \subseteq \text{EXP}. \quad (34)$$

§3. NP Completeness

“一个合格的计算机科学家能直觉地判断一个问题是否可计算。”

作为一个开始,先撇开NP的猜测定义,而给粗一个等价的定义:

Definition 3.1.

$L \subseteq \{0,1\}^*, L \in \text{NP}, \iff \exists p: \mathbb{N} \mapsto \mathbb{N}, M$ 是多项式时间的图灵机, 且 $\forall x \in \{0,1\}^*,$ 下述等价关系成立: $x \in L \iff \exists u \in \{0,1\}^{p(|x|)}. M(x, u) = 1$

我们称 M 是 L 的验证器. 若 $|u| = |p(x)|$, 称 u 是 x 的证书.

Remark. L 视为一类问题, x 就是这类问题的一个具体的实例, u 可以视为 x 的解.

如果对每个 x (具体的问题), 都有这样的关系: 一个输入 x 属于问题集合 L , 当且仅当 存在一个 u , 它的长度不超过 $p(|x|)$, 并且验证器 M 拿着 x 和 u 一跑, 而且这个计算的时间还是多项式时间. $\square$

Proof. 考虑通用图灵机对非确定图灵机的模拟, 和非确定图灵机的猜测能力来考察 iff 的两边. $\square$

我们可以理解为: P 是能在多项式时间找出证明的问题, 而 NP 是多项式长证明的问题. 下面是一些 NP -问题的例子.

Example.

1. 独立集 IS
2. 旅行商 TSP
3. 图同构问题 GI

§3.1. NP-难

先来定义 Karp 规约, 因为我们想刻画 NP 里面的很难的问题.

Definition 3.1.1.

$L, L' \subseteq \{0,1\}^*$. 称 $L \leq_K L'$, 如果有一个函数 $r: \{0,1\}^* \mapsto \{0,1\}^*$, 使得对于所有 $x \in \{0,1\}^*, x \in L \iff r(x) \in L'$.

称 L 是 L' 的 Karp 规约.

Definition 3.1.2.

L 是 NP -难的, $\forall A \in NP$, 使得 $A \leq_K L$. 进一步, 如果 $L \in NP$, 则 L 是 NP -完全的.

令人不禁想问(或者只有傅想问:)真的有 NP 完全问题吗? 有一个简单的思路: 构造所有的 NP 问题的问题. 记作 TMSAT 问题.

Definition 3.1.3 (有界停机问题).

$\text{TMSAT} = \{ \langle \alpha, x, 1^n, 1^t \rangle \mid \text{存在 } u \in \{0,1\}^n, \text{使得图灵机 } M_\alpha \text{ 在输入 } (x, u) \text{ 上运行 } t \text{ 步后输出 } 1 \}$

它是 NP -完全的. 证明如下:

Proof.

Step1. $\text{TMSAT} \in NP$

拿到一个证书 u (长度 n) 模拟图灵机 M_α 在输入 (x, u) 上运行 t 步 如果它输出 $1 \rightarrow$ 接受; 否则拒绝 模拟 t 步的时间是 $O(t)$, 而 t 是输入的一部分, 所以输入长度至少是 t , 因此模拟时间是多项式级别的.

所以 $\text{TMSAT} \in \text{NP}$.

Step2. $\exists L \in \text{NP}, \text{TMSAT} \leq_K L$

我们记 $L = \{\langle \alpha, x, 1^n, 1^t \rangle \mid \text{存在 } u \in \{0, 1\}^n, \text{使得健忘图灵机 } U_\alpha \text{ 在输入 } (x, u) \text{ 上运行 } t \text{ 步后输出 } 1\}$. 则 $L \leq_K \text{TMSAT}$, 取

$$f(z) = \begin{cases} \langle \alpha, x, 1^n, 1^{c(|\alpha|)t \log t} \rangle, & z = \langle \alpha, x, 1^n, 1^t \rangle \\ z, & \text{else} \end{cases} \quad (35)$$

所以我们分两步证明了命题. □

Theorem 3.1.1.

If $P = \text{NP}$ then $\text{EXP} = \text{NEXP}$

Proof. 说白了是添加垃圾. 若 $L \in \text{NEXP}$ 在 2^{n^c} 时间内被计算, 那么, 用垃圾信息填充:

$$L^* = \{x01^{2^{|x|^c}} \mid x \in L\} \in \text{NP} \quad (36)$$

把每个输入 x 后面加上一串很长的 01 串, 长度是 $2^{|x|^c}$ (指数级长). 现在 L^* 已经属于 NP 了. 那么根据假设, $L^* \in P \Rightarrow L \in \text{EXP}$. □

Remark. 读者应当把这个添加垃圾信息与后面的 Ladner 定理扯上关系. □

§3.2. Cook-Levin 定理

回顾一下合取范式 CNF. $k\text{CNF}$ 是最多 k 个变量的 CNF.

在第一章我们就学过 SAT, 它是所有可满足的 CNF 的集合. 2-SAT 是指每一个语句最多只有 2 个变量的主合取范式.

Theorem 3.2.1.

SAT 是 NP-完全的.

Proof. 先来对 SAT 有一些认知.

Lemma 3.2.1.

$\text{SAT} \leq_K 3\text{-SAT}$. 考虑: $u_1 \vee u_2 \vee u_3 \vee u_4 \vee u_5 = (u_1 \vee u_2 \vee v) \wedge (\neg v \vee u_3 \vee w) \wedge (\neg w \vee u_4 \vee u_5)$.

反之也成立 (这是因为 3-SAT 是 SAT 的子集). 所以, 3SAT 和 SAT 是一样困难的.

Step1. 准备工作

注意, 我们可以把一堆等号表示成合取范式: $x = y \Leftrightarrow (x \vee y) \wedge (\neg x \vee \neg y)$.

断言: $\forall f: \{0, 1\}^l \mapsto \{0, 1\}$, 我们有一个 (最多) $l2^l$ -CNF 的合取范式 φ_f , 使得 $f(x) = \varphi_f(x)$. 也就是说, 可以为任意布尔函数 f 构造一个 CNF 公式 φ_f , 让它和 f 在所有输入上输出完全一致.

断言是正确的. 这是因为, 对于一个指派, 我们总能在 l 个符号内把 $f(x)$ 的值表示出来. 又有 2^l 个指派.

定义 $L \in \text{NP}$. 由本文开篇提到的定义, $L \in \text{NP}$ 等价于: 存在通用多项式图灵机 M 和多项式 p 使得 $\forall x \in \{0, 1\}^*, x \in L \Leftrightarrow \exists u \in \{0, 1\}^{p(|x|)}, M(x, u) = 1$.

我们来构造一个多项式时间可计算的 $\varphi: L \rightarrow \text{SAT}$, s.t. $\varphi(x) \in \text{SAT} \Leftrightarrow x \in L$. 也就是说构造一个 Krap 归约,

$$\varphi(x) \in \text{SAT} \iff \exists u \in \{0, 1\}^{p(|x|)}, \mathbb{M}(x, u) = 1. \quad (37)$$

Step2. 计算的逻辑刻画

本步的主要动机是想把图灵机的计算变成公式. 设单带图灵机 $\mathbb{M} = (\Gamma, Q, \delta)$, 时间函数 $T(n), t = T(|x|)$. 不妨假设终止格局 C_t 时, 带子上只是由输入、结果(0/1)、 $\square$ 组成的, 并且读写头指向那个结果(这是因为读写头只动了 t 步). 具体的, 如图所示.

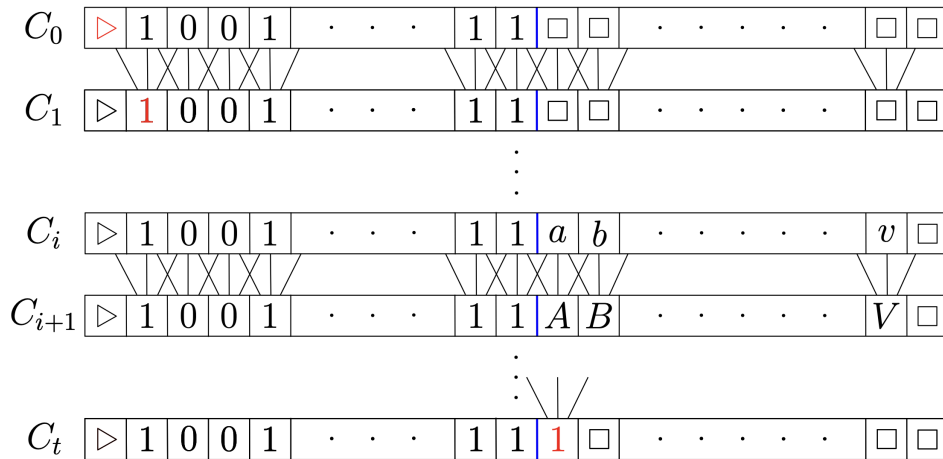


Figure 7: 哦

我们将格局 C_i 的带子上的每一个符号用 $l \triangleq \log(|\Gamma|) + \log(|Q|) + 1$ 长度的 01 串编码. 三个项的意思: 第一个是存符号, 第二个是存格局的状态; 最后一个存第 i 时刻读写头是否指向本格.

图中的 A 之所以为 A, 是因为它会依赖于三个位置: 上一时刻它顶上的 a, 或者 a 左边的 1, 或者 a 右边的 b.

那么, 立刻可以用 $f: \{0, 1\}^{3l} \mapsto \{0, 1\}^l$ 来计算 $C_i[j]$. 从上面三个的编码, 可以推出下面一个的编码.

这等价于可以用 $\bar{f}: \{0, 1\}^{4l} \mapsto \{0, 1\}$ 来等价表示 (Why? 类比 $x \wedge y \wedge z \rightarrow w$ 和利用蕴含表达式之后的结果). 这是一个布尔函数, 根据断言, 有一个合取范式 ϕ . 如果记 $z^{i,j} = C_i[j]$, 则 $\bar{f}$ 就是

$$\phi_{i,j} \triangleq \phi(z^{i-1,j-1}, z^{i-1,j}, z^{i,j-1}, z^{i,j}) \quad (38)$$

的一堆公式 $\phi_{i,j}$ 构成的.

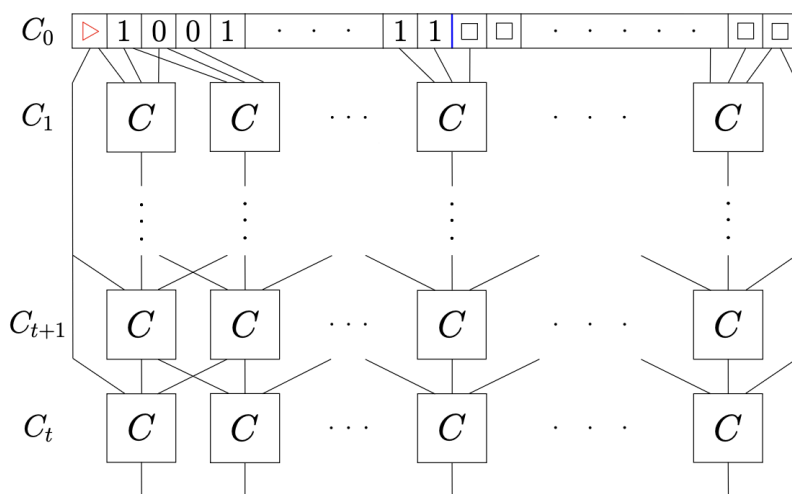


Figure 8: 图里的那么多 C 就是之前提到的 $\bar{f}$. 这么一堆公式合取出来就得到了输出.

最后定义 φ 为所有局部字句的合取. 这是确保计算历史合法: 在 CNF 公式中, 所有的 $C_{i,j}$ 都是“未赋值的布尔变量”, SAT 求解器可以任意给它们赋值——只要满足局部子句. 如果不强制它们之间的时间一致性, 就可能出现逻辑上合法但计算上不可能的赋值.

注意,每一个 C 的大小不超过 $4l^{2^4l}$. l 依赖于图灵机本身的定义,所以 C 的大小是常量. 而一共有 t^2 个 C , 所以输出的长度是 $O(t^2)$, 是多项式时间的.

这就证明了归约所花费的时间是多项式时间的. □

Karp 随后提出了 21 个 NP-完全问题, 让这个研究变得很有意义. (也让找到一个 NP-完全问题这种 idea 发不了论文——傅育熙)

Example (Karp 的 21 个问题).

- SAT
 - 0-1 Integer Programming
 - Clique
 - Set Packing
 - Vertex Cover
 - Set Covering, Feedback Node Set, Feedback Arc Set
 - Directed Hamilton Circuit
 - Undirected HC
 - 3-SAT
 - Chromatic Number
 - Clique Cover
 - Exact Cover
 - Hitting Set, Steiner Tree, 3-Dimensional Matching
 - Knapsack
 - Job Sequencing
 - Partition
 - Max Cut

Problem 3.2.1 (Godel's Question).

设 $\mathcal{A}$ 是一个公理系统, 该公理系统可以解释合取范式,

THEOREM = $\{\varphi, 1^{|\varphi|^c}\} | \varphi$ 有一个长度不超过 $|\varphi|^c$ 的 $\mathcal{A}$ 中的证明

Proof.

1. 显然, 给定一个证明, 计算机能在“证明长度”的多项式时间内, 快速验证它是否正确.
2. 我是区. □

§3.3. Berman-Hartmanis 猜测

Berman-Hartmanis 猜测: 所有 NP-完全问题都是等价的, 也就是, 多项式同构的,

$$\exists \text{ Cook-Levin reductions } L, A \leq_L B \leq_L A \quad (39)$$

那么, 记作 $A \cong_p B$. 我不想看 Myhill 自同构定理, 但是他们证明了 $\exists \text{ Karp reductions } k, k^{-1}, \Rightarrow A \cong_p B$.

Theorem 3.3.1.

Berman-Hartmanis 猜测 $\Rightarrow P \neq NP$.

Proof. 对语言 $S \subseteq \{0, 1\}^*$, $S^{\leq n}$ 表示 S 中所有长度不超过 n 的串的集合. 我们称 S 是稠密的, iff, $|S^{\leq n}| = 2^{n^{O(1)}}$; S 是稀疏的, iff, $|S^{\leq n}| = n^{O(1)}$.

Lemma 3.3.1.

记 L 稠密, L' 稀疏.则 $L \cong_p L'$ 不成立.

Proof. 反证,设 r 是双射卡普规约并且在多项式时间($p(n)$)计算 $L \rightarrow L'$.则 $r(L(n)) \subseteq L'^{\leq p(n)}$.但是 L 稠密,所以 $|L'^{\leq p(n)}| = 2^{O(p(n))}$.所以 $L'^{\leq p(n)}$ 是稠密的,矛盾. $\square$

熟知 SAT 是稠密的,则其不与 $\{1^n \mid n \in \mathbb{N}\}$ 同构.若 $P = NP$,则 $NP_C = P$,则 $\{1^n \mid n \in \mathbb{N}\}$ 是 NP 完全的.矛盾!这就证明了 $P \neq NP$. $\square$

Remark.

1. recall that NP_C 是 NP 完全的问题构成的集合,而 Karp 规约是构造一个时间可计算函数把一个问题打到另一个问题上.
2. Why SAT is dense? 考察

$$\phi = (x_1 \vee x_2 \vee x_3) \bigvee_i C_i \quad (40)$$

其中, x_i 让 ϕ 变得恒真,然后我们随便选 C_i 使得 $|\phi| = n$.这至少是 2^n 的.

3. NP_C 的意思是 NP -完全问题. $\square$

§3.4. Ladner 定理

令人不禁想问(其实只有傅和甘想问), $NP \setminus P$ 里面有其他问题吗?Ladner 定理证明了, $NP \neq P$ 时, $NP \setminus P$ 里面恰好有无穷个其他问题.接下来考察一个弱化的版本.

Theorem 3.4.1.

若 $NP \neq P$, $NP_C \cup P \neq NP$.

Proof. 证明的动机是, $SAT \in NP$ 完全的,但是 $2SAT \in P$.想扔掉 SAT 里的一些元素,让他不 NP 完全,但是又不至于落到 P 里面.

利用填充技术,将 $\phi \iff \phi \wedge n_1 \wedge \dots \wedge n_h$,这里的等价符号的意思是可满足的等价性.用二进制,就表示为 $\phi 01^h$.

容易验证, $\{\phi 01^{|\phi|^c} \mid \phi \in SAT\}$ 是 NP 完全的,而 $\{\phi 01^{2^{|\phi|}} \mid \phi \in SAT\}$ 是 P 的(直接暴力枚举所有变元的值即可).

记

$$SAT_H = \{\phi 01^{|\phi|^{H(|\phi|)}} \mid \phi \in SAT\} \quad (41)$$

并定义示性函数 $SAT_H(x)$.

我们如何做 H ?来做对角线法.对角化思想:如果某个算法想“猜中” SAT_H ,我们就改变 SAT_H 的结构,让它失败.

$$H(n) = \begin{cases} i, & i < \log(\log(n)) \wedge (\forall x, M_i(x) \text{ 在 } |x|^i \text{ 步内输出 } SAT_H(x)) \\ \log(\log(n)), & \text{else} \end{cases} \quad (42)$$

如果存在这样的 i ,则令 $H(n) = i$,这意味着我们发现了一个可能的 P 算法(在小输入上表现良好).于是我们“惩罚”它:把 $H(n)$ 设为 i ,从而让填充量变小(因为 i 很小),使得 SAT_H 更难,破坏这个算法的正确性.如果没发现,我们就往上跳.

$H(n)$ 有两个性质:

1. $H(n)$ 是非递减函数.
2. H 可在多项式时间内计算.

计算步骤就和定义一样,枚举 i ,然后枚举确定的通用图灵机 M_i ,然后模拟那么多步计算.

- 根据通用图灵机的定义,模拟一个图灵机在一个输入 x 上时间为 $c(\log(\log(\log(n))))C \log C$, $C = \log(\log(n))(\log(n))^{\log(\log(n))}$ (这是 $i|x|^i$ 直接代入的结果).

带入之前的可得到 $c(\log(\log(\log(n))))C \log C$ 是 $o(n)$ 的,这是因为下标小于 $\log(\log(n))$ 的自然数的二进制长度不会超过三个 $\log$.

- 我们最多需要检查 $\log \log n$ 个图灵机,每个图灵机要检查所有满足 $|x| \leq \log(n)$ 的 $|x|$,这样的 x 有 $2^{\log(n)} = O(n)$ 个(外面的 $O(n)$).

使用暴力算法,判定 $x \in SAT_H$ 的时间是 $2^{\log(n)} = O(n)$ 的(括号里面的那个 $O(n)$).

同时,我们需要递归的看 $x = \phi 01^{|\phi|^{H(|\phi|)}}$ 里的 $\phi \in H$. 而 $|\phi| = o(\log(|x|))$ 的. 所以,

$$\begin{aligned} T(n) &\leq (\log \log(n))O(n)(o(n) + O(n) + T(\log(n)) + O(1)) \\ \Rightarrow T(n) &= o(n^3). \end{aligned} \quad (43)$$

(直接从上式好像不好看出为啥是 $o(n^3)$ 的. 但我们可以待定一下系数,例如 $T(n) = o(n^k) \leq o(n^k)$, 通过归纳法确定 k . 事实上 $T(n)$ 比较接近 $n^2 \text{ poly}(n)$, 这里单纯只是为了证明比较方便)

现在来用反证法证明 $SAT_H \notin P$.

假设 $SAT_H \in P$, 则存在 M_i 在 cn^c 时间内计算 SAT_H . 则对充分大的 n , $H(n) \leq i$. 又 H 不递减, 说明 $\exists n_0, \forall n > n_0, H(n) = D$ 为常数.

这意味着 $SAT \leq_K SAT_H$ (结合 SAT_H 的定义), 这说明 $SAT \in P$, 又 SAT 是 NP -完全的, 故 $P = NP$, 与前提矛盾.

再来用反证法证明 $SAT_H \notin NPC$.

若 $SAT_H \in NPC$, 则存在一个在 dn^d 时间内计算的规约 $r: SAT \mapsto SAT_H$. 那么, $H(n)$ 是没有上界的, 否则和上述证明一样了, 矛盾.

那么, 由上述, $\exists N, |\psi| > d \wedge$

$$|r(\varphi)| = |\psi 01^{|\psi|^{H(|\psi|)}}| > N \wedge H(|\psi|) > 2d + 1 \quad (44)$$

. 则

$$\begin{aligned} |\psi|^{2d+1} &< |\psi|^{H(|\psi|)} < |r(\varphi)| \leq d|\varphi|^d < |\psi||\varphi|^d \\ \Rightarrow |\psi| &< \sqrt{|\varphi|}. \end{aligned} \quad (45)$$

所以, 我们构造 $Sat(\varphi)$ 算法如下:

1. 计算 $|\varphi|$ 为 $\psi 01^{|\psi|^{H(|\psi|)}}$ 形式;
2. 若 $r(\varphi) > N$, 递归 $Sat(\psi)$;
3. 否则, 直接算.

设调用了 k 次, 则 $1 \leq |\varphi|^{2^{-k}} \leq N$. 则 $k \leq \log(\log(|\varphi|))$. 所以多项式时间可计算, $SAT_H \in P$. 这又是一个矛盾.

□

§3.5. 神谕图灵机, 以及更多

§3.5.1. 神谕图灵机的定义、Lowness

我们想主观的解决所有的“子程序调用”问题, 这样引入了神谕的概念.

Definition 3.5.1.1 (Oracle Turing Machine).

一个神谕图灵机 $M^?$ 有额外的一条读写神谕带和状态 $q_{\text{query}}, q_{\text{yes}}, q_{\text{no}}$.

- $M^?$ 需要一个判定问题, 也就是神谕 $B \subseteq \{0, 1\}^*$.

- 在执行 M^B 时, 每当 M^B 进入状态 q_{query} , 机器移动到状态 q_{yes} 如果 $a \in B$ 和 q_{no} 如果 $a \notin B$, 其中 a 是神谕带上的内容.
- 一个查询到 B 算作单步计算. 换言之, 无论 B 有多难, 查询只花 1 步时间.

我们用 $M^{B(x)}$ 表示 M^B 在输入 x 上的输出.

一类神谕可以用该类中最难的问题作为代表, 例如: $\text{NP}^{\text{NP}} = \text{NP}^{\text{SAT}}$.

Definition 3.5.1.2 (Lowness).

若 $A^B = A$, 称复杂性类 B 对复杂性类 A 是低的.

Remark.

- 直观理解, 有没有神谕 B 对 A 而言都一样, 也就是说 B 对 A 而言过于简单. (就像计算理论对 YYL 过于简单)
- P 、 L 、 NL 对自己而言是低的 (第一个是显然的, 后面两个是因为空间可复用), EXP 对自己 ertain 不是低的.
- 普遍认为 NP 对自己而言不是低的. 也就是说, $\text{NP}^{\text{NP}} \neq \text{NP}$.

□

我们可以利用哥德尔编码来给神谕图灵机编码并排序 $M_0^?, M_1^?, \dots$

我们还能用神谕图灵机来定义一些复杂的复杂性类. 例如, P^O 是带神谕 O 的确定图灵机可判定的问题所构成的集合.

我们还能用这些复杂性类来定义复杂性类. 例如(作业里已经有了)

$$\text{NP}^{\text{NP}} \triangleq \bigcup_{O \in \text{NP}} \text{NP}^O. \quad (46)$$

$\text{NP}^{O[k]}$ 是 NP^O 的一个子类.

§3.5.2. Cook Reduction

Definition 3.5.2.1 (Cook Reduction).

若 $A \in \text{P}^B$, 称 A 可 Cook 归约到 B , 记为 $A \leq_C B$.

L 可 Cook 归约到 L' iff L 可 Cook 归约到 $\overline{L'}$.

TODO: 详细地解释一下 Cook 归约 (多项式时间图灵归约) 允许在归约过程中多次调用神谕, 因此它不是“一对一”的映射; 而 Karp 归约 (多项式时间多归约) 是“一对一”的, 因为它只允许一次转换、不允许多次查询.

我们来看(finally!)本章的最后一个定理:

Theorem 3.5.2.1 (Baker-Gill-Solovay Theorem).

$\exists A, B, \text{P}^A = \text{NP}^A \wedge \text{P}^B \neq \text{NP}^B$.

Proof.

- 第一个 case: $\text{PSPACE} = \text{P}^{\text{PSPACE}} \in \text{NP}^{\text{PSPACE}} = \text{PSPACE}$. 让神谕 A 足够强大, 以至于 P 机器也能“模拟” NP 机器. 任意 PSPACE 完全的问题都能解决问题, 例如 QBF.
- 第二个 case: 直观地想, 如果有一个问题, 拒绝它需要调用大于多项式次数的神谕, 所以我们在调用多项式次数的情形下会犯错, 那么构造这样的问题就好了. 不妨设这样的神谕是 B , 并且有 $B_0 \subseteq B_1 \subseteq B_2 \dots, B = \bigcup B_i$. 并同时 $n_0 < n_1 < n_2 \dots$. 这里 n_{i+1} 只和调用神谕的时候询问 $a \in B_i$ 的 a 有关. 有可能 a 非常长, 但我们只需要 n_{i+1} 比所有可能的 a 更长.

我们的目标是:对这个语言 $L_B \triangleq \{1^n \mid B \text{ 包含一个长度为 } n \text{ 的 } 01 \text{ 串}\}$, 我们构造 B , 让所有多项式图灵机 M_i^B 犯错, 也就是 $L_B \notin \mathbf{P}^B$. ($L_B \in \mathbf{NP}^B$, 是因为非确定图灵机总能猜测这个 s , 然后调用神谕, 而不是光靠枚举所有可能的 s)

利用先前提到的哥德尔编码, 我们想让 $M_i^{B_i}$ 无法正确判断 $1^{n_{i+1}}$ 是否属于 B .

归纳地构造 B_i :

1. 如果 $M_i^{B_i}$ 在 $2^{n_{i+1}-1}$ 步内判定 $1^{n_{i+1}}$ 为拒绝, 那我们就让 $B_{i+1} = B_i + s, s \in \{0, 1\}^{n_{i+1}}$.
2. 如果 $M_i^{B_i}$ 在 $2^{n_{i+1}-1}$ 步内判定 $1^{n_{i+1}}$ 为接受, 或者没停, 那我们就让 $B_{i+1} = B_i$.

这两步以来, B 里就是一系列“稀疏的”字符串集合, 只在特定的 i 下有一个该长度的字符串.

选择 n_{i+1} 足够大, 保证 $M_i^{B_i}$ 在 $2^{n_{i+1}-1}$ 步内只能查询 B_i 中的 $\text{poly}(n_{i+1})$ 个字符串 (因为每一步最多查一个字符串, 而 B_i 只包含之前加入的有限个字符串). 因此, 我们可以安全地加入一个 s , 它不在 $M_i^{B_i}$ 的查询范围内, 从而不影响 $M_i^{B_i}$ 在 B_i 上的行为.

$\mathbf{NP}$ 机器可以非确定性地猜出正确的 s , 然后查询神谕, 所以它总能正确判断 $1^{n_{i+1}}$ 是否属于 L_B .

根据对角线法则, 我们可以发现 L_B 就是所求. 命题得证.

3. 证明 $L_B \in \mathbf{NP}^B, L_B \notin \mathbf{P}^B$. 前者已经在证明的细节里提到了. 后者: 反证法. 如果 M_i^B 在 $T(n)$ 时间内判定 L_B , 取 i 充分大, 一定有 $T(n_i) < 2^{n_i-1}$. 按定义,

$$\begin{aligned} M_i^B(1^{n_{i+1}}) = 0 &\iff \forall x \in B, |x| \neq n_{i+1} \\ &\iff \forall x \in B_{i+1}, |x| \neq n_{i+1} \\ &\iff M_i^{B_{i+1}}(1^{n_{i+1}}) = 1 \\ &\iff M_i^B(1^{n_{i+1}}) = 1 \end{aligned} \tag{47}$$

矛盾. □

Remark.

1. 为什么 $\mathbf{NP}^{\mathbf{PSPACE}} = \mathbf{PSPACE}$? 对一个 $N^{\mathcal{O}}, \mathcal{O} \in \mathbf{PSPACE}$, 它非确定的猜测一个 $\text{poly}(n) + \text{poly}(T(\mathcal{O}))$ 路径长度的可能分枝. 所以, 对一个 $\mathbf{PSPACE}$ 图灵机 M_P , 可以由 dfs 的方法来模拟所有可能分支.
2. 注意一下, 其实 $|\{0, 1\}^n| = 2^n > n^k$, 所以光靠枚举是枚举不完所有的字符串的, 这也是一个我们能让多项式图灵机犯错的基本逻辑.
3. 动机: 因为每一步最多查一个字符串, 而 B_i 只包含之前加入的有限个字符串, 但是增大 n_{i+1} 之后查询的次数也增大了.

提问: 理论上, 是不是可以每次加入一大堆 s , 或者在 B_0 里加入一堆垃圾语言, 严重延缓 $\mathbf{P}$ 图灵机的枚举?

回答: 这是没弄清楚 $\mathbf{P}$ 判定该语言的方式. 一个 $\mathbf{P}$ 图灵机无法感知 B 里面有什么, 只能一个个枚举 n 长度的 01 串是否存在于 B 内, 再怎么更改 B 也没用, 加入垃圾无从谈起.

□